SDA WEEK 5

Syed Ahmed Khan

syed.ahmed@nu.edu.pk

RECAP

Week 3: **Use case:** Use cases are functional requirements They describe what the system does from the user's perspective. They are about behavior, sequences of actions, and achieving a specific goal for an actor. For example, 'Borrow Book', 'Withdraw Cash', 'Register for Course'.

Week 4: **Class Diagram:** Class diagrams are a static structure of the system in question. They are the blueprint. They show the classes, their attributes, their operations (or methods), and the relationships between them, like associations, generalizations, and aggregations.

Use-Case Realization and Robustness Analysis.

This week lectures would attempt to find out, how does the behavior described in a use case get implemented by the structure defined in the class diagram.

USE CASE REALIZATION

- Use-Case Realization is the process of designing how the classes and objects in our system will collaborate to perform the behavior described in a use case.
- use-case realization describes how a particular use case is realized within the Design Model, in terms of collaborating objects.

USE CASE REALIZATION

- The use case says 'the system shall dispense cash'. Realization answers: 'Which object initiates the action? Which object holds the account balance? Which object physically commands the cash dispenser? What messages do they send to each other?

USE CASE REALIZATION

A REAL WORLD ANALOGY

- The use case is the script for a play.
- The class diagram is the list of actors and props.
- Use-Case Realization is the process of blocking the scene–deciding which actor says which line, when they move, and how they interact with the props.

USE CASE REALIZATION

To model these collaboration or realizations, two primary types of diagrams may be used are:

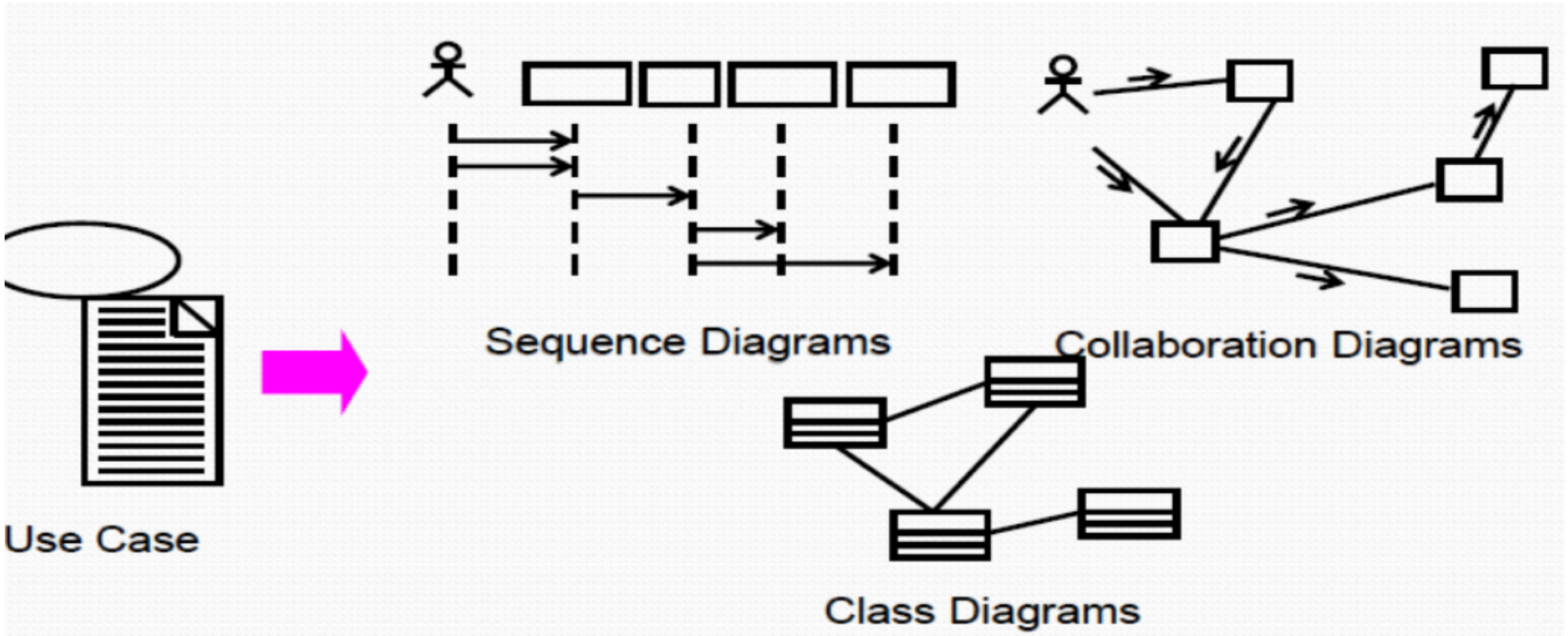
INTERACTION DIAGRAMS

Sequence/Communication diagrams: These can be used to describe how the use case is realized in terms of collaborating objects.

CLASS DIGRAMS

Class Diagrams can be used to describe the classes that participate in the realization of the use case, as well as their supporting relationships

USE CASE REALIZATION



WHO DOES USE CASE REALIZATION

- **Designer**: They must ensure the integrity of the entire realization, coordinating with other designers responsible for individual classes. This is a team effort where responsibilities are distributed.

- The use-case realization can be used by class designers to understand the class’s role in the use case and how the class interacts with other classes.

ROBUSTNESS ANALYSIS

Robustness Analysis is the technique of analyzing the text of a use case description, sentence by sentence, and identifying a first-guess set of the participating objects, classifying them as Boundary, Control, or Entity.

The outcome of a robustness analysis is an analysis class.

ANALYSIS CLASS

Analysis classes represent an early conceptual model of the system. They are a rough draft, a sketch. They are fluid and will likely change–many will morph into subsystems, be split, or be combined. Their purpose is not to be perfect but to capture the required behaviors in a form we can use to explore the system's composition.

We find them by asking three questions:

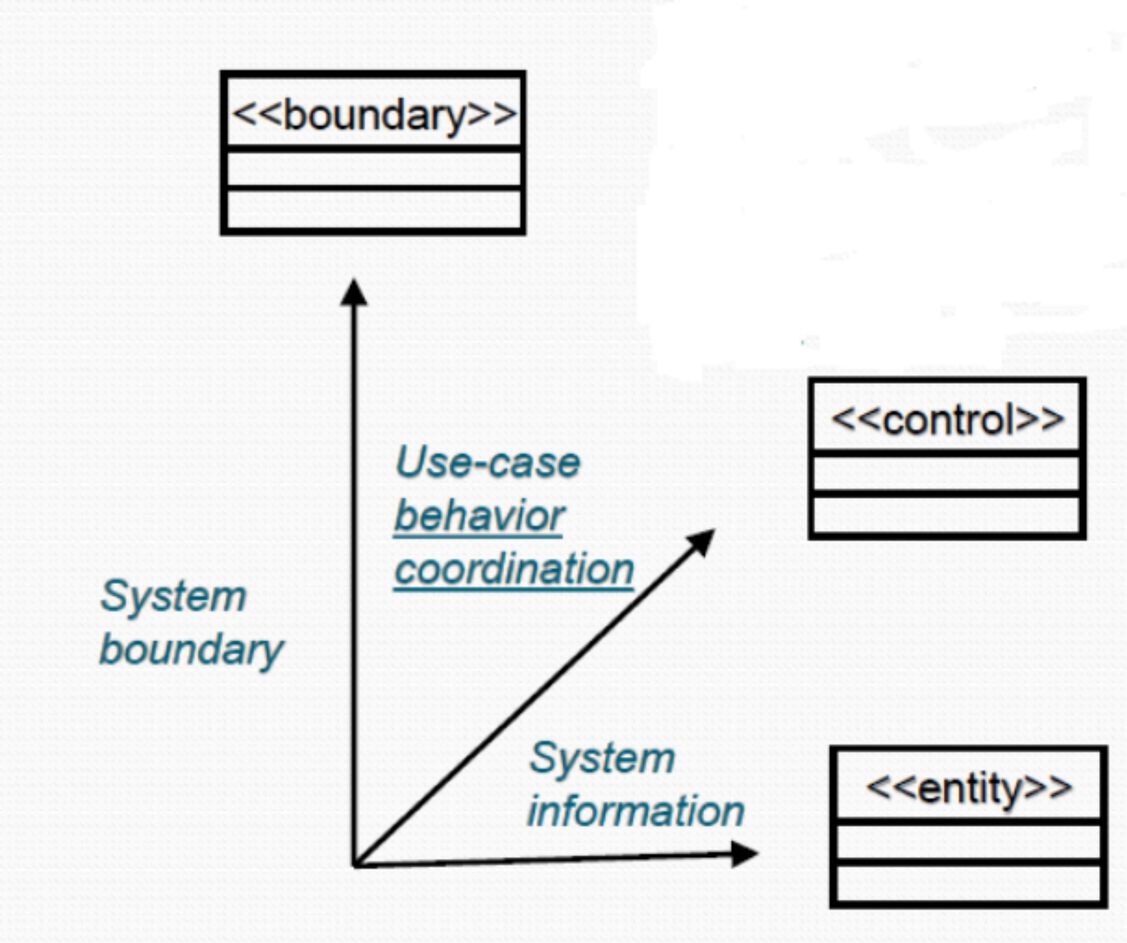
1. What’s the boundary with the user or other systems?
2. What information do we need to store or manage?
3. What control logic ties things together?

ANALYSIS CLASS

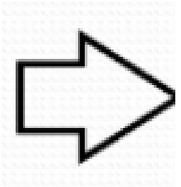
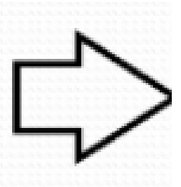
We need a way to categorize these early classes. We do this using three powerful stereotypes, which are like labels that tell us the primary purpose of a class during analysis:

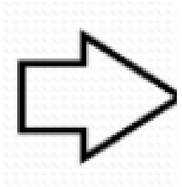
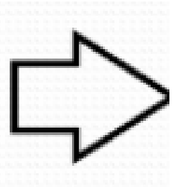
1. **<<boundary>>:** Classes that sit on the boundary between the system and its external actors.
2. **<<control>>:** Classes that coordinate the flow of logic for a use case.
3. **<<entity>>:** Classes that represent persistent information or key domain concepts.

ANALYSIS CLASS



ANALYSIS CLASS

Use cases  Analysis classes  Design elements

 Source Code  Executables

BOUNDARY CLASS

- Boundary objects are responsible for supporting communications between the system's external environment and its internal workings (i.e., control and entity objects).
- Within the context of use case realization, there will be one boundary class for each user interface.
- The actor(s) identified within the Use Case Model will always interact with the system through these boundary objects
- Boundary classes insulate the system from changes outside. For instance, if you redesign your app's user interface, your business logic remains unaffected because the boundary class absorbs the change.

BOUNDARY CLASS

Types of boundary classes:

- User interfaces (forms, windows, web pages).
- System interfaces (communication with other systems or databases).
- Device interfaces (sensors, hardware inputs).

BOUNDARY CLASS

- **Rule**: Actors can only communicate with boundary classes. They define the system's edge.

Example: In a student registration system, CourseRegistrationForm could be a boundary class

CONTROL CLASS

- Control classes hold the application-specific business logic. They don’t store data, but they orchestrate operations. Each use case often has a corresponding control class.
- These are the 'glue' between boundaries and entities.

CONTROL CLASS

- Within the context of use case realization, each boundary class will communicate with a single control class and control classes will be used to manage each use case’s flow of execution.

- To manage this flow, the control object must coordinate the activities required to support the use case realization, including interactions with other control objects and the data aware entity objects.

CONTROL CLASS

Example: A control class, RegisterStudentControl might:

- Get input from RegistrationForm (boundary),
- Validate it,
- Update Student and Course (entities),
- And finally, return confirmation to the boundary.

ENTITY CLASS

- Entity classes are persistent. They represent real-world concepts stored in the system. They live on beyond one use case.
- They are typically used to represent the key items the system manages.
- Entity objects (instances of entity classes) are used to hold and update information about some phenomenon, such as an event, a person, or some real-life object.

1. They are usually persistent, having attributes and relationships needed for a long period, sometimes for the life of the system.
2. The main responsibilities of entity classes are to store and manage information in the system.

ENTITY CLASS

- To display the data, the data encapsulated within the entity object will traverse a path that eventually leads to the control object that is tightly coupled to the boundary object

- At this point, the data will be passed to the boundary object for displaying in the GUI

ANALYSIS CLASS

To summarize:

- Analysis Classes are our first step towards executable code.
- They are a conceptual model built using three key stereotypes–Boundary, Control, Entity that help us organize our thinking and create a resilient design.
- Next time, we will see how these elements are forbidden from talking to each other and use this knowledge to perform Robustness Analysis.

THE ECB DESIGN PATTERN AND ROBUSTNESS ANALYSIS

MVC & ECB

Previously, we have introduced three analysis class stereotypes: Boundary, Control, and Entity. Now, we'll place them into a formal pattern and see the rules that govern their interactions.

ECB is based on a refinement of a classic design pattern called Model-View-Controller (MVC), which dates back to early 80s.

MVC & ECB

MVC's goal is to decompose an application into these distinct parts to promote separation of concerns.

- **Model**: The data and business logic (maps to our <<entity>> classes).
- **View**: The presentation layer (maps to our <<boundary>> classes).
- **Controller**: Handles input and controls flow (maps to our <<control>> classes).

Our Entity-Control-Boundary (ECB) pattern is a direct descendant of MVC, adapted for use-case driven analysis and design with specific communication rules.

ECB DESIGN PATTERN COMMUNICATION RULES

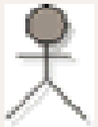
The power of the ECB pattern isn't just in the classification; it's in the strict rules that govern how these elements can communicate. These rules are what force a good, decoupled design.

ELEMENTS OF ECB

ALLOWED COMMUNICATIONS

- An Actor can only talk to a Boundary object. (e.g., A user clicks a button on a form).
- A Boundary object can talk to a Control object. (e.g., The form sends the clicked event to a controller).
- A Control object can talk to Entity and other Control objects. (e.g., The controller asks an Entity for data to validate).
- An Entity object can talk to other Entity objects. (e.g., A Student object checks its association with Course objects).

ALLOWED COMMUNICATIONS

Actor or Object Initiating Communication	Flow of Communication	Target Object
		
		
		
		

COMMUNICATIONS NOT ALLOWED

- An Actor cannot talk directly to a Control or Entity object. They must go through a Boundary.
- A Boundary object cannot talk directly to an Entity object. All business logic must be routed through a Control object. This prevents UI code from being riddled with complex business rules.
- An Entity object should not talk to a Boundary or Control object. It should be passive, waiting to be used.

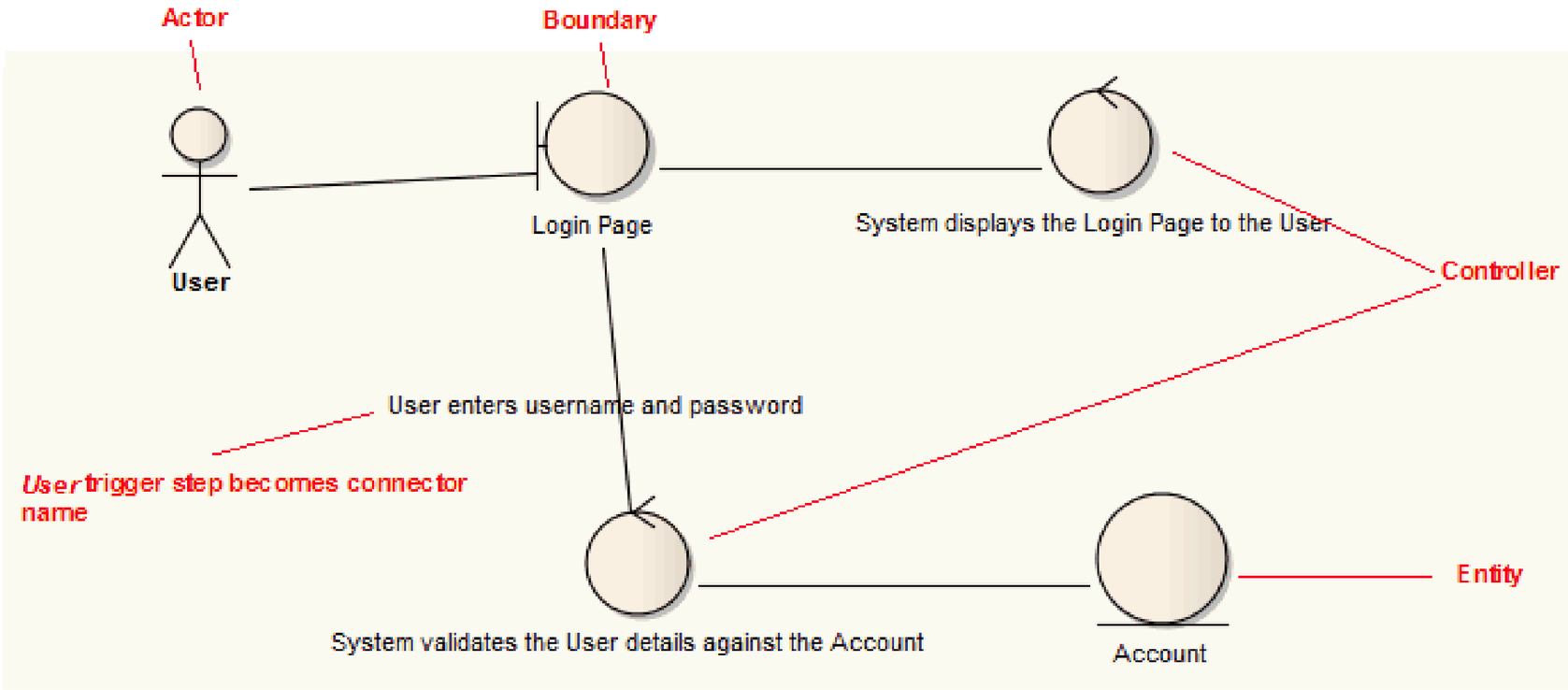
COMMUNICATIONS NOT ALLOWED

Actor or Object Initiating Communication	Flow of Communication	Target Object
		
		
		
		
		

ROBUSTNESS ANALYSIS EXAMPLE

Step	Action
 1	System displays the Login Page to the User
 2	User enters username and password
 3	System validates the User details against the Account

ROBUSTNESS DIAGRAM



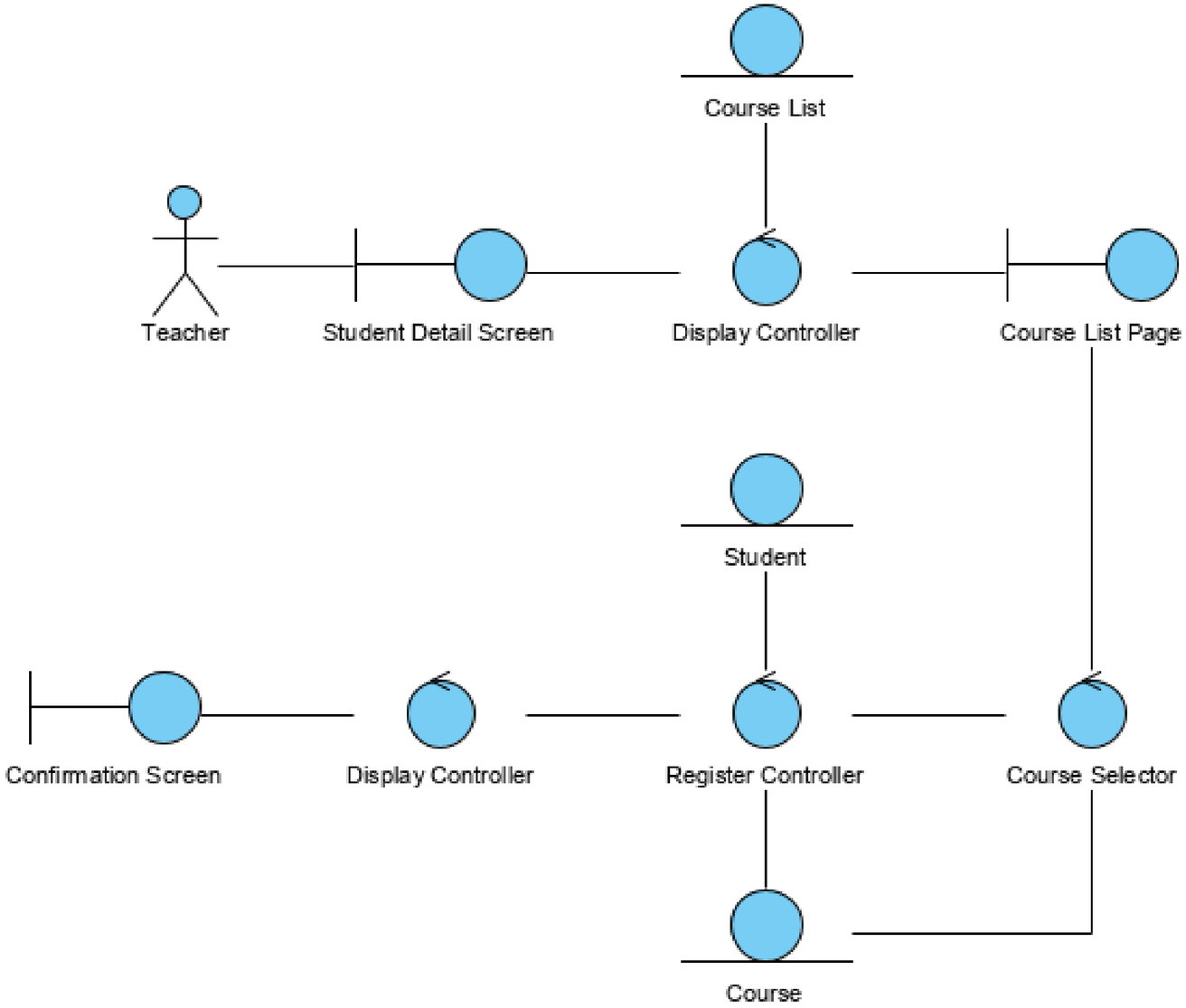
EXERCISE

From the student detail page, the teacher clicks on the ‘‘Add courses’’ button and the system displays the list of courses. The teacher selects the name of a course and presses the ‘‘Register’’ button. The system registers the student for the course. And displays the confirmation message on Confirmation screen.

EXERCISE

- What are the actors? (Teacher)
- What are the boundary classes? (StudentDetailPage, CourseListPage, ConfirmationScreen)
- What is the key control logic? (The process of registering a student for a course. This suggests a RegisterControl class)
- What entity classes are involved? (Student, Course

EXERCISE

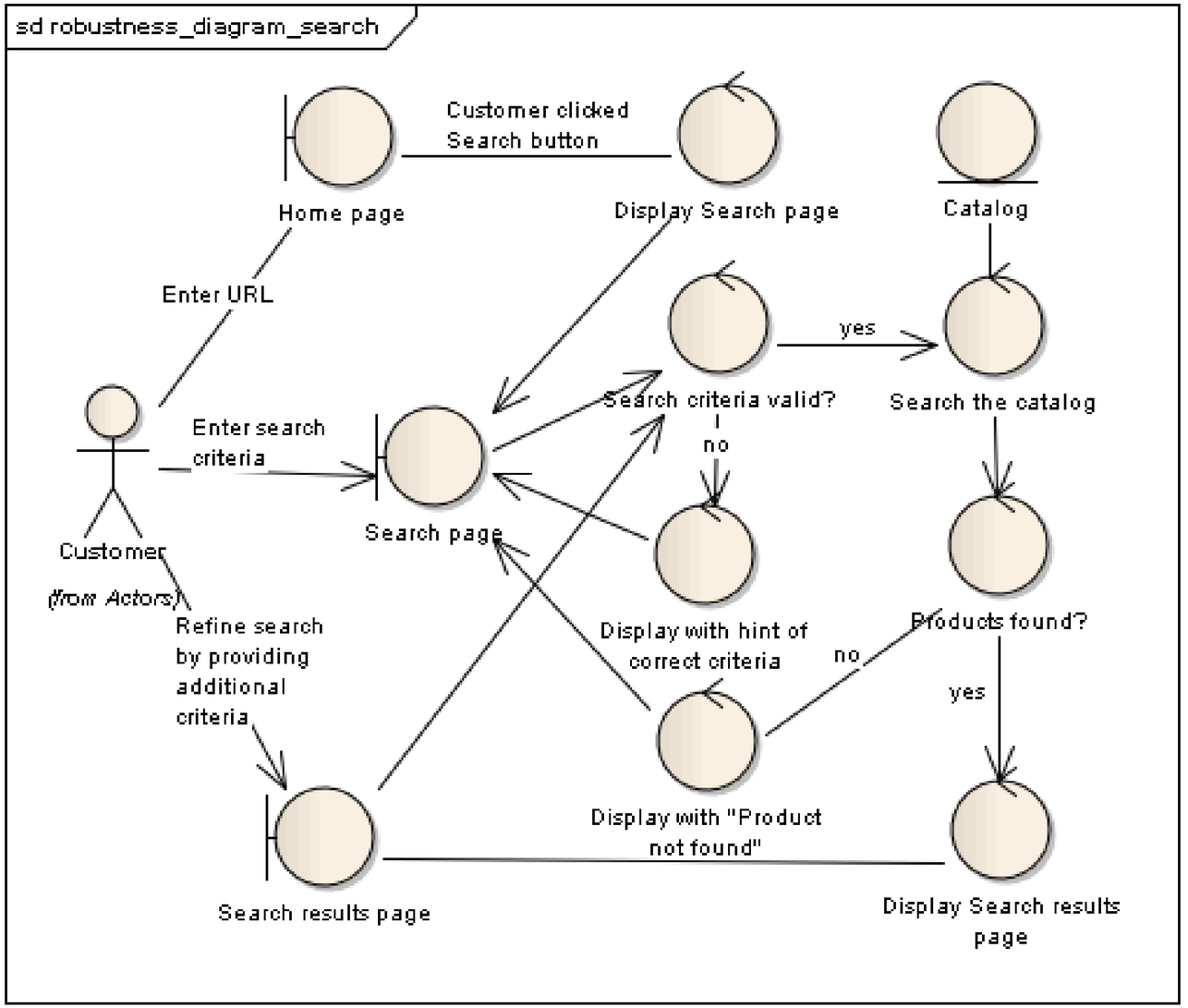

Course List
Teacher
Student Detail Screen
Display Controller
Course List Page
Student
Confirmation Screen
Display Controller
Register Controller
Course Selector
Course

EXERCISE

Element	Type	Responsibility
Teacher	Actor	Initiates the use case by interacting with the UI.
Student Detail/Course List/Confirmation Screens	Boundary	What the user sees and interacts with.
Display/Register Controllers, Course Selector	Control	The "brains" that coordinate the action, enforce rules, and manage
Student, Course, (Teacher as data)	Entity	The core data objects of the system that are updated and manipulated.

Step	Actor / Action	Boundary Object	Control Logic	Entity Object	Outcome / Purpose
1	**Teacher** views student details.	**Student Detail Screen** is displayed.		**Student**, **Teacher**	The teacher is on a screen showing a specific student's information.
2	**Teacher** clicks "Add Courses" or similar.	**Student Detail Screen** captures the click event.			The teacher initiates the registration action.
3			**Display Controller** is activated.		The system prepares to show the available courses.
4			**Display Controller** fetches course data.	**Course**	The control logic retrieves the list of available courses from the entity.
5		**Course List Page** is displayed.	**Display Controller** populates the list.		The interface now shows the teacher the catalog of
6	**Teacher** selects a course and clicks "Register".	**Course List Page** captures the selection and			The teacher specifies which course to register
7			**Register Controller** is activated.		The main coordination logic for the enrollment process begins.
8			Register Controller may invoke	**Course**	The controller may use a helper to validate the selection (e.g., check for
9			**Register Controller** updates student record.	**Student**	The core business logic executes: the student entity is modified to enroll in the course.
10			**Register Controller** updates course roster.	**Course**	The course entity is modified to add the student to its roster.
11			**Register Controller** confirms success.		The enrollment transaction is completed successfully.
12		**Confirmation Screen** is displayed.			The user interface provides visual feedback that the action was successful.
13	**Teacher** sees the confirmation.				The use case is complete.

EXERCISE#2: SEARCH PRODUCT



EXERCISE#2: SEARCH PRODUCT

Step	Actor / Action	Boundary Object	Control Logic	Entity Object	Outcome / Decision
1	**Customer** enters a URL.	**Home Page** is displayed.			The process begins.
2	**Customer** enters search criteria.	**Search Page** captures the input.			Data is ready for processing.
3			**Validate Search Criteria**		Check if the input is valid.
4					**Is criteria valid?** → Decision Point
4a					**No:** Display a hint for correct criteria. Process ends.
4b					**Yes:** Proceed to **Search the catalog**.
5			**Search the Catalog**	**Catalog**	The system queries the product database.
6			**Check if Defaults Found?**		Checks if the search returned any results.
7					**Were results found?** → Decision Point
7a					**No:** Display "Product not found" message.
7b			**Refine Search**		**Yes:** The system may automatically refine the search with additional
8		**Search Results Page**			The final results are prepared for display.
9		**Display Search Results Page**			The results are shown to the customer. Process completes.

THANK YOU